Northern University of Bussiness and Technology Khulna

MIT $(^w^)$

Sourav Mondal, Rahul Kumar Ghosh, Ayon Adikary Arko

adapted from KACTL and MIT NULL

2025-10-16

1 Contest 1

2 Mathematics 2

3 Data Structures 2

Contest (1)

template.cpp

103 lines

```
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
using db = long double; // or double, if TL is tight
using str = string; // yay python! //

// pairs
using pi = pair<int,int>;
using pl = pair<ll,ll>;
using pd = pair<db,db>;
#define mp make_pair
#define f first
#define s second

#define tcT template<class T
#define tcTU tcT, class U
// ^ lol this makes everything look weird but I'll try it
tcT> using V = vector<T>;
tcT, size_t SZ> using AR = array<T,SZ>;
using vi = V<int>;
using vb = V<bool>;
using vl = V<ll>;
using vd = V<db>;
using vs = V<str>;
using vpi = V<pi>;
using vpl = V<pl>;
using vpd = V<pd>;

// vectors
// oops size(x), rbegin(x), rend(x) need C++17
#define sz(x) int((x).size())
#define bg(x) begin(x)
#define all(x) bg(x), end(x)
#define rall(x) x.rbegin(), x.rend()
#define sor(x) sort(all(x))
#define rsz resize
#define ins insert
#define pb push_back
#define eb emplace_back
#define ft front()
#define bk back()

#define lb lower_bound
#define ub upper_bound
tcT> int lwb(V<T>& a, const T& b) { return int(lb(all(a),b)-bg(
    ↪a)); }
tcT> int upb(V<T>& a, const T& b) { return int(ub(all(a),b)-bg(
    ↪a)); }

// loops
#define FOR(i,a,b) for (int i = (a); i < (b); ++i)
#define F0R(i,a) FOR(i,0,a)
#define ROF(i,a,b) for (int i = (b)-1; i >= (a); --i)
#define R0F(i,a) ROF(i,0,a)
#define rep(a) F0R(_,a)
#define each(a,x) for (auto& a: x)

const int MOD = (int)1e9+7; // 998244353;
const int MX = (int)2e5+5;
const ll BIG = 1e18; // not too close to LLONG_MAX
const db PI = acos((db)-1);
const int dx[4]{1,0,-1,0}, dy[4]{0,1,0,-1}; // for every grid
    ↪problem!!
mt19937 rng((uint32_t)chrono::steady_clock::now().
    ↪time_since_epoch().count());
template<class T> using pqg = priority_queue<T,vector<T>,
    ↪greater<T>>;

// bitwise ops
// also see https://gcc.gnu.org/onlinedocs/gcc/Other-Builtins.
    ↪html
constexpr int pct(int x) { return __builtin_popcount(x); } // #
    ↪ of bits set
constexpr int bits(int x) { // assert(x >= 0); // make C++11
    ↪compatible until USACO updates ...
  return x == 0 ? 0 : 31-__builtin_clz(x); } // floor(log2(x))
constexpr int p2(int x) { return 1<<x; }
constexpr int msk2(int x) { return p2(x)-1; }

ll cdiv(ll a, ll b) { return a/b+((a^b)>0&&a%b); } // divide a
    ↪by b rounded up
ll fdiv(ll a, ll b) { return a/b-((a^b)<0&&a%b); } // divide a
    ↪by b rounded down

tcT> bool ckmin(T& a, const T& b) {
  return b < a ? a = b, 1 : 0; } // set a = min(a,b)
tcT> bool ckmax(T& a, const T& b) {
  return a < b ? a = b, 1 : 0; } // set a = max(a,b)

tcTU> T fstTrue(T lo, T hi, U f) {
  ++hi; assert(lo <= hi); // assuming f is increasing
  while (lo < hi) { // find first index such that f is true
    T mid = lo+(hi-lo)/2;
    f(mid) ? hi = mid : lo = mid+1;
  }
  return lo;
}
tcTU> T lstTrue(T lo, T hi, U f) {
  --lo; assert(lo <= hi); // assuming f is decreasing
  while (lo < hi) { // find first index such that f is true
    T mid = lo+(hi-lo+1)/2;
    f(mid) ? lo = mid : hi = mid-1;
  }
  return lo;
}
tcT> void remDup(vector<T>& v) { // sort and remove duplicates
  sort(all(v)); v.erase(unique(all(v)),end(v)); }
tcTU> void erase(T& t, const U& u) { // don't erase
  auto it = t.find(u); assert(it != end(t));
  t.erase(it); } // element that doesn't exist from (multi)set

int main() { cin.tie(0)->sync_with_stdio(0); }
```

.bashrc

3 lines

```
alias clr="printf '\33c'"
co() { g++ -std=c++17 -O2 -Wall -Wextra -Wshadow -Wconversion -
    ↪o $1 $1.cpp; }
run() { co $1 && ./$1; }
```

hash.sh

3 lines

```
# Hash file ignoring whitespace and comments. Verifies that
# code was correctly typed. Usage: 'sh hash.sh < A.cpp'
cpp -dD -P -fpreprocessed|tr -d '[:space:]'|md5sum|cut -c-6
```

stress.sh

22 lines

```
#!/usr/bin/env bash

for ((testNum=0;testNum<$4;testNum++))
do
    "./$3".out > input
    "./$2".out < input > outSlow
    "./$1".out < input > outWrong
    H1=`md5sum outWrong`
    H2=`md5sum outSlow`
    if !(cmp -s "outWrong" "outSlow")
    then
        echo "Error found!"
        echo "Input:"
        cat input
        echo "Wrong Output:"
        cat outWrong
        echo "Slow Output:"
        cat outSlow
        exit
    fi
done
echo Passed $4 tests
```

troubleshoot.txt

75 lines

```
General:
Write down most of your thoughts, even if you're not sure
whether they're useful.
Give your variables (and files) meaningful names.
Stay organized and don't leave papers all over the place!
You should know what your code is doing ...

Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Remove debug output.
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output as well.
Read the full problem statement again.
Have you understood the problem correctly?
Are you sure your algorithm works?
Try writing a slow (but correct) solution.
Can your algorithm handle the whole range of input?
Did you consider corner cases (ex. n=1)?
Is your output format correct? (including whitespace)
Are you clearing all data structures between test cases?
Any uninitialized variables?
Any undefined behavior (array out of bounds)?
Any overflows or NaNs (or shifting ll by >=64 bits)?
Confusing N and M, i and j, etc.?
Confusing ++i and i++?
Return vs continue vs break?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some test cases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Rewrite your solution from the start or let a teammate do it.

Geometry:
Work with ints if possible.
```

```
Correctly account for numbers close to (but not) zero. Related:
for functions like acos make sure absolute val of input is not
(slightly) greater than one.
Correctly deal with vertices that are collinear, concyclic,
coplanar (in 3D), etc.
Subtracting a point from every other (but not itself)?

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What's your complexity? Large TL does not mean that something
simple (like NlogN) isn't intended.
Are you copying a lot of unnecessary data? (References)
Avoid vector, map. (use arrays/unordered_map)
How big is the input and output? (consider FastIO)
What do your teammates think about your algorithm?
Calling count() on multiset?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?
If using pointers try BumpAllocator.
```

Mathematics (2)

2.1 Trigonometry

$$\sin(v+w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v+w) = \cos v \cos w - \sin v \sin w$$
$$\tan(v+w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v+w}{2} \cos \frac{v-w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v+w}{2} \cos \frac{v-w}{2}$$

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

2.2 Geometry

2.2.1 Triangles

Side lengths: a, b, c

Semiperimeter: $s = \dfrac{a+b+c}{2}$

Area: $A = \sqrt{s(s-a)(s-b)(s-c)}$

Circumradius: $R = \dfrac{abc}{4A}$

Inradius: $r = \dfrac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$$

Law of sines: $\dfrac{\sin\alpha}{a} = \dfrac{\sin\beta}{b} = \dfrac{\sin\gamma}{c} = \dfrac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\dfrac{a+b}{a-b} = \dfrac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

2.3 Derivatives/Integrals

$$\frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx}\tan x = 1 + \tan^2 x \qquad \frac{d}{dx}\arctan x = \frac{1}{1+x^2}$$
$$\int \tan ax = -\frac{\ln|\cos ax|}{a} \qquad \int x \sin ax = \frac{\sin ax - ax\cos ax}{a^2}$$
$$\int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) \qquad \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1)$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.4 Sums/Series

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \le 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \le x \le 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

Data Structures (3)

3.1 STL

MapComparator.h

Description: example of function object (functor) for map or set

Usage: `set<int,cmp> s; map<int,int,cmp> m;`

5bfa6c, 1 lines

```
struct cmp{bool operator()(int l,int r)const{return l>r;}};
```

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th element, and finding the index of an element. Change `null_type` to get a map.

Time: $\mathcal{O}(\log N)$

<ext/pb_ds/assoc_container.hpp>, <ext/pb_ds/assoc_container.hpp> 59e4dd, 18 lines

```
using namespace __gnu_pbds;
template<typename T> using Tree = tree<T, null_type, less<T>,
  ↪rb_tree_tag, tree_order_statistics_node_update>;
#define ook order_of_key
#define fbo find_by_order

using namespace __gnu_pbds;
template<typename T, typename C>
using IndexedSet = tree<T, null_type, less<T, rb_tree_tag,
  ↪tree_order_statistics_node_update>;

void treeExample() {
  Tree<int> t, t2; t.insert(8);
  auto it = t.insert(10).f; assert(it == t.lb(9));
  assert(t.ook(10) == 1 && t.ook(11) == 2 && *t.fbo(0) == 8);
  t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

3.2 1D Range Queries